

Compte-rendu de TP3 : Mastermind Réseau

Programmation de Sockets TCP en Java

Ismael Ahamada mmadi

Février 2026

Table des matières

1	Introduction	2
2	Architecture Réseau et Protocole	2
2.1	Fonctionnement de TCP	2
3	Implémentation du Serveur	2
4	Gestion des Erreurs (Améliorations)	2
5	Conclusion	2

1 Introduction

L'objectif de ce TP est l'implémentation d'un jeu de Mastermind en réseau. Contrairement à l'UDP qui traite des datagrammes indépendants, nous utilisons ici le protocole **TCP** (Transmission Control Protocol) pour établir un flux de données fiable entre un **Codeur** (Serveur) et un **Décodeur** (Client).

2 Architecture Réseau et Protocole

2.1 Fonctionnement de TCP

Le choix de TCP garantit que chaque proposition de couleur envoyée par le client arrive au serveur sans perte et dans l'ordre exact. Le processus suit le cycle suivant :

1. **Établissement** : Connexion via le port 1234.
2. **Échange** : Le client envoie une chaîne de 4 caractères ; le serveur répond avec deux entiers (bien placés et mal placés).
3. **Fermeture** : La connexion est rompue une fois la victoire atteinte (score 4 0).

3 Implémentation du Serveur

Le serveur a été conçu pour être **multi-threadé**. Cela permet de lever la limitation du serveur simple : plusieurs clients peuvent jouer simultanément sans s'attendre.

```
1 while (true) {  
2     Socket clientSocket = serverSocket.accept();  
3     // Chaque client est g r dans un thread s par  
4     new Thread(new GameHandler(clientSocket)).start();  
5 }
```

Listing 1 – Extrait du Serveur Multi-thread

4 Gestion des Erreurs (Améliorations)

Conformément à l'étape 1.6 du TP, le code gère les situations suivantes :

- **Taille de saisie** : Si la chaîne reçue ne fait pas exactement 4 caractères, le serveur renvoie un message d'erreur.
- **Déconnexion** : Le serveur détecte la fin du flux si le client ferme brutalement sa socket, évitant ainsi une exception fatale.

5 Conclusion

Ce TP a permis de valider la compréhension des sockets en mode connecté. La séparation entre la logique métier (classe **Code**) et la couche réseau (TCP) permet une grande modularité. Le passage au multi-threading est essentiel pour toute application réseau destinée à plusieurs utilisateurs.